UNIVERSITÉ FRANÇAISE KEYCE INFORMATIQUE

ET INTELLIGENCE ARTIFICIELLE

RAPPORT DE L'ATELIER DE CYBERSECURITE

BACHELOR 2

ATELIER 2

PRÉSENTATION TENUE DU 01 AU 22 Avril 2025

RÉDIGÉ ET PRÉSENTÉ PAR LES MEMBRES DU GROUPE 1

SOUS LA DIRECTION ACADÉMIQUE DE :

M. ESSOMBA

ANNEE ACADEMIQUE 2024/2025

INTRODUCTION

Dans le cadre de ce projet pédagogique et professionnel, il a été demandé de concevoir et de sécuriser un environnement réseau virtuel destiné à simuler l'infrastructure d'une entreprise fictive nommée **Cyber Limited**. L'objectif principal était de mettre en place un **Proof of Concept (POC)** complet afin de tester des solutions de sécurité couramment utilisées en entreprise. Pour ce faire, cinq machines virtuelles ont été déployées sous VMware Workstation : un pare-feu pfSense, deux systèmes Linux (Ubuntu Server et Ubuntu Client) ainsi que deux systèmes Windows (Windows Server 2019 et Windows 10). Chaque machine a été configurée avec une adresse IP statique sur un réseau interne isolé. Le projet a consisté à installer et paramétrer pfSense comme pare-feu central, tenter l'intégration d'un IDS (Snort), appliquer des mesures de durcissement sur chaque système (pare-feu, restriction de services, sécurité SSH, GPO, fail2ban, antivirus, etc.), puis à effectuer des **tests de sécurité concrets** tels que des scans de ports, des ping floods ou des tentatives de connexions SSH par force brute afin de vérifier l'efficacité des protections mises en place. L'ensemble des étapes a été documenté, illustré par des captures d'écran, et analysé dans une logique de mise en situation réaliste, proche des pratiques professionnelles en cybersécurité.

I. PRESENTATION DE L'ENVIRONNEMENT DE TEST

1. Outils utilisés

Pour mettre en place l'environnement de test, l'outil de virtualisation **VMware Workstation** a été utilisé. Ce choix permet de créer plusieurs machines virtuelles (VM) sur un seul ordinateur physique, avec une gestion fine du réseau et des ressources.

Voici les outils et systèmes utilisés :

- **VMware Workstation** : pour créer, connecter et gérer les VM.
- **pfSense** (pare-feu basé sur FreeBSD) : pour contrôler le trafic réseau.
- **Ubuntu Server 22.04** : système Linux sans interface graphique, utilisé pour héberger des services.
- **Ubuntu Desktop 22.04** : utilisé comme poste client Linux.
- **Windows Server 2019** : pour simuler un serveur d'entreprise avec des rôles administratifs.
- **Windows 10** : pour simuler un poste utilisateur classique.

Le réseau a été entièrement isolé du réseau Internet via une carte réseau virtuelle personnalisée (**VMnet2**), simulant un environnement d'entreprise interne sécurisé.

2. Schéma réseau

L'ensemble des machines est connecté à un **réseau interne virtuel** VMnet2.
La machine pfSense agit comme **passerelle et pare-feu** entre toutes les autres machines.
Chaque machine possède une **adresse IP statique** et communique uniquement via pfSense.

3. Adressage IP des machines

Machine	Système	Rôle	Adresse IP
pfSense	pfSense (FreeBSD)	Pare-feu & passerelle	192.168.10.1
Ubuntu Client	Ubuntu Desktop 22.04	Poste utilisateur Linux	192.168.10.10
Ubuntu Server	Ubuntu Server 22.04	Serveur Linux sécurisé	192.168.10.11

Windows Server	Windows Server 2019	Serveur Windows sécurisé	192.168.10.12
Windows 10	Windows 10	Poste utilisateur	192.168.10.13

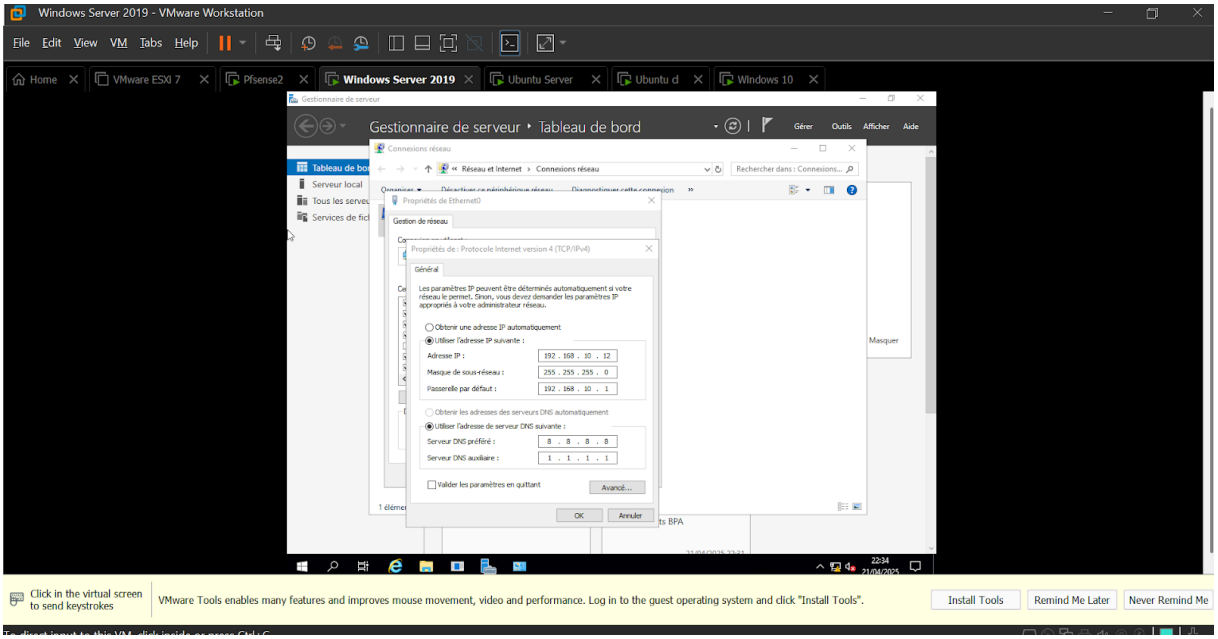

Figure 1 : Configuration de l'adresse IP sur Windows Server

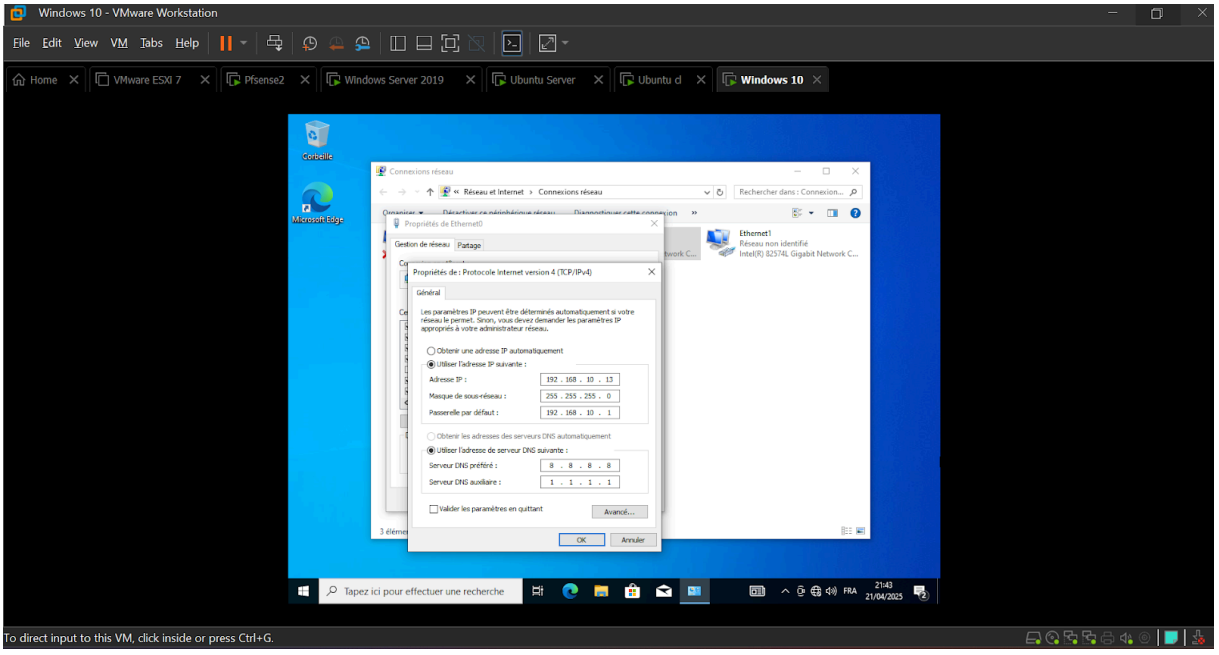

Figure 2 : Configuration de l'adresse IP sur Windows 10

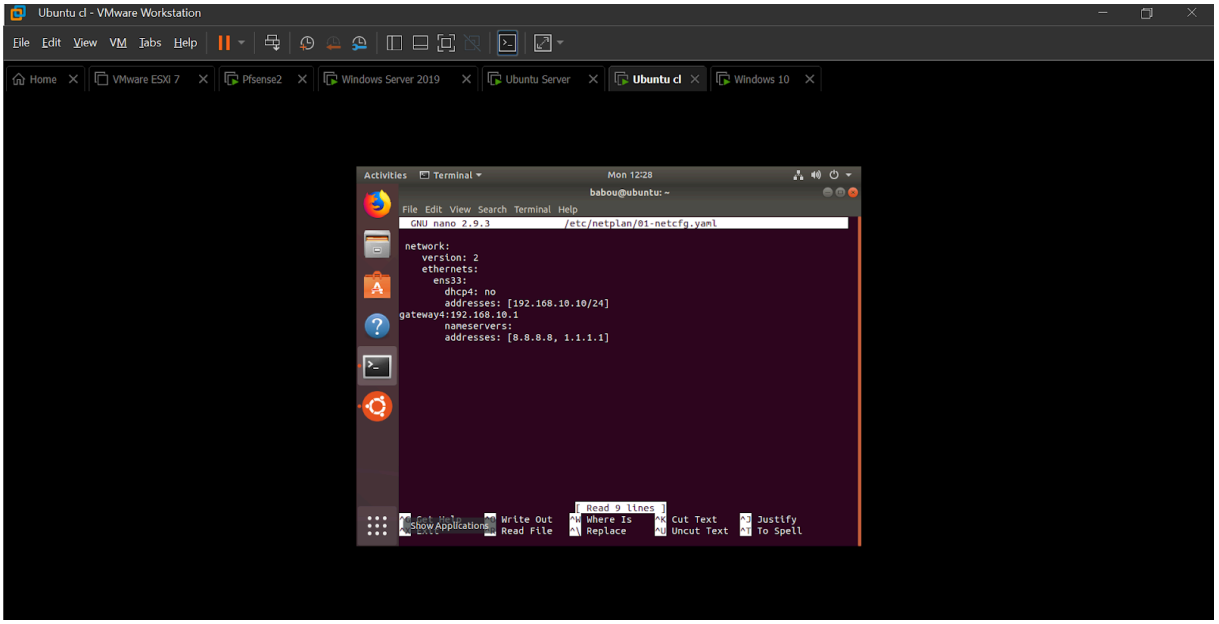

Figure 3 : Configuration de l'adresse IP sur Ubuntu Client

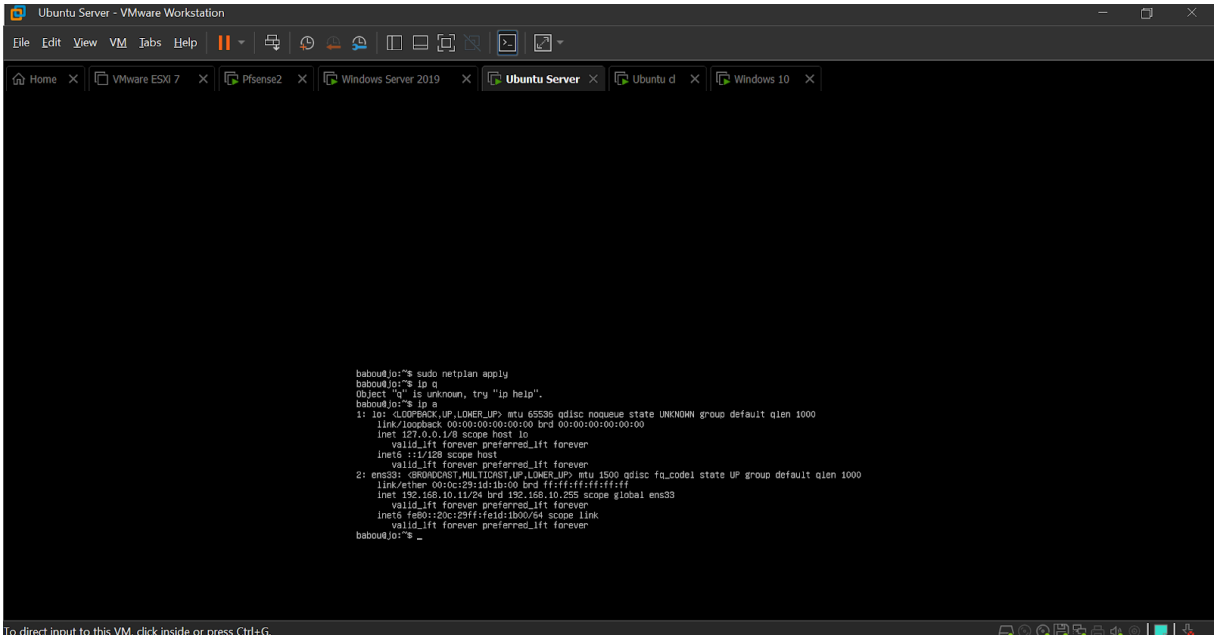

Figure 4 : Configuration de l'adresse IP sur Ubuntu Server

4. Rôle de chaque machine virtuelle

- **pfSense** : C'est le **cœur de la sécurité réseau**. Il filtre les connexions entre les machines via des règles de pare-feu, contrôle le routage, et peut héberger des outils comme un IDS.

- **Ubuntu Server** : Machine utilisée pour expérimenter des configurations serveur, installer des outils comme Snort, et tester le durcissement Linux (UFW, fail2ban, sécurité SSH).
- **Ubuntu Client** : Poste utilisateur typique Linux sur lequel des mesures de sécurité sont appliquées (pare-feu local, suppression de services, nettoyage des utilisateurs).
- **Windows Server 2019** : Serveur simulant un environnement professionnel Windows avec des stratégies de sécurité (GPO), pare-feu avancé, gestion des utilisateurs.
- **Windows 10** : Poste client Windows classique, configuré avec les règles de sécurité d'usage (antivirus, pare-feu, verrouillage, contrôle d'accès local).

II. CONFIGURATION DU PARE-FEU AVEC PFSENSE

1. Introduction à pfSense et rôle dans l'infrastructure

PfSense est une distribution open source basée sur FreeBSD, spécialisée dans la sécurité réseau. Elle est largement utilisée dans les environnements professionnels comme **pare-feu, routeur, point d'accès VPN**, et même comme **plateforme de détection d'intrusion**. Dans ce projet, pfSense joue un rôle **central** : il fait office de **passerelle sécurisée** entre toutes les machines internes et agit comme **point unique de contrôle du trafic**.

Son intégration dans l'environnement de Cyber Limited répond à plusieurs objectifs :

- Protéger le réseau local contre toute intrusion
- Filtrer les flux réseau entre les machines
- Tester la configuration d'un pare-feu professionnel
- Observer et contrôler les communications

2. Installation de pfSense sur VMware

L'installation de pfSense a été réalisée en important l'image ISO officielle dans une nouvelle machine virtuelle créée sur VMware Workstation. Deux interfaces réseau ont été configurées dans les paramètres de la VM :

- Une interface **WAN** connectée en mode **NAT**, simulant l'accès à Internet
- Une interface **LAN** connectée au réseau **VMnet2**, permettant de relier toutes les autres machines

Pendant l'installation, pfSense a automatiquement détecté les deux interfaces :

- **em0** pour le WAN (Internet/NAT)
- **em1** pour le LAN (réseau interne 192.168.10.0/24)

L'installation s'est déroulée sans encombre en suivant les étapes classiques : partitionnement automatique, installation de base, reboot et lancement en mode console.

3. Configuration des interfaces et du réseau LAN

Une fois pfSense redémarré, la configuration initiale a été réalisée via la **console texte**. Les interfaces ont été attribuées comme suit :

- **WAN (em0)** : configurée en DHCP (obtenue automatiquement depuis le NAT VMware)
- **LAN (em1)** : configurée en IP statique 192.168.10.1/24

Cette adresse LAN devient la **passerelle par défaut** pour toutes les autres machines virtuelles connectées à VMnet2.

À ce stade, pfSense était déjà capable de :

- Joindre toutes les machines internes (Ubuntu, Windows)
- Être pingé depuis ces machines
- Jouer son rôle de routeur entre les sous-réseaux

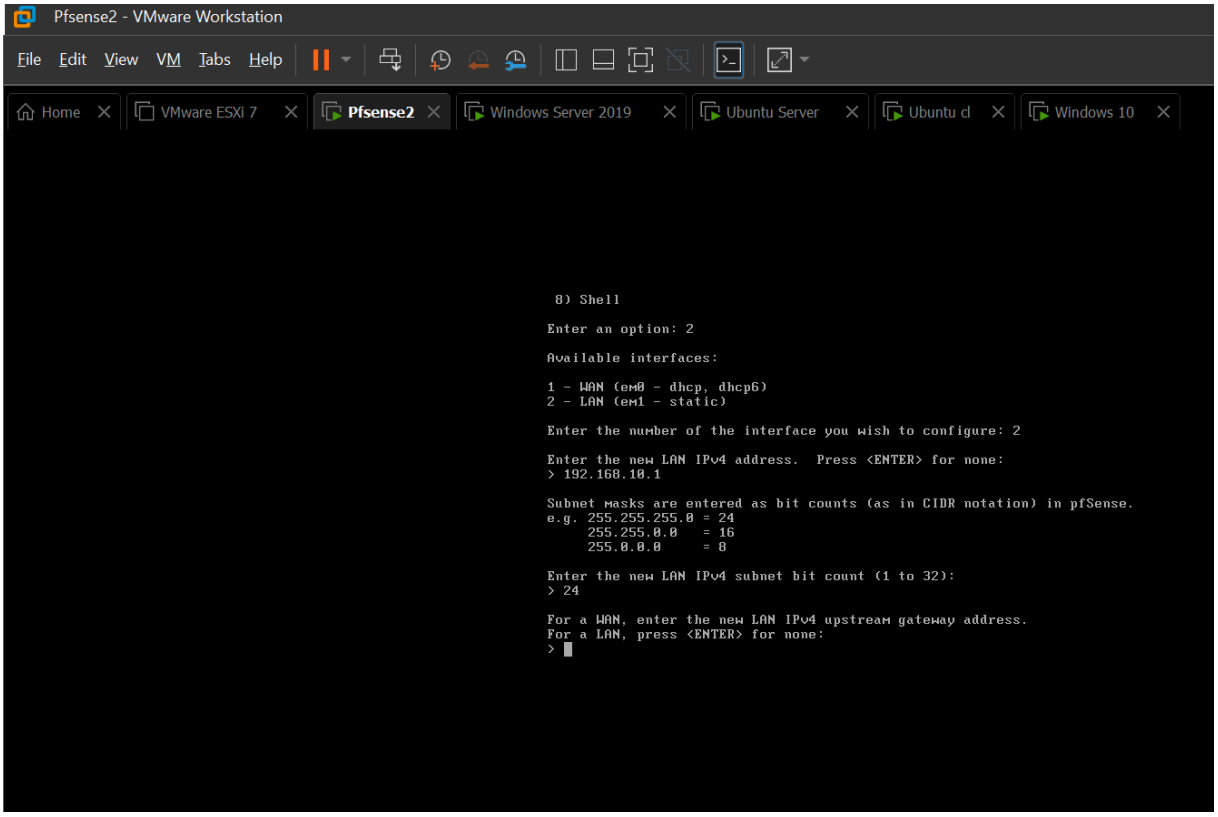

Figure 5 : Configuration de l'adresse IP statique de pfsense

4. Accès à l'interface Web – WebConfigurator

Une fois les interfaces configurées, un navigateur sur Ubuntu Client a été utilisé pour accéder à l'interface Web de pfSense à l'adresse :

https://192.168.10.1

Comme il s'agit d'un certificat auto-signé, le navigateur a affiché un avertissement de sécurité. Celui-ci a été ignoré pour accéder à l'interface (comportement courant en environnement local sécurisé).

Après connexion avec les identifiants par défaut :

- **Utilisateur :** admin
- **Mot de passe :** pfsense

L'assistant **de configuration initiale** s'est lancé automatiquement.

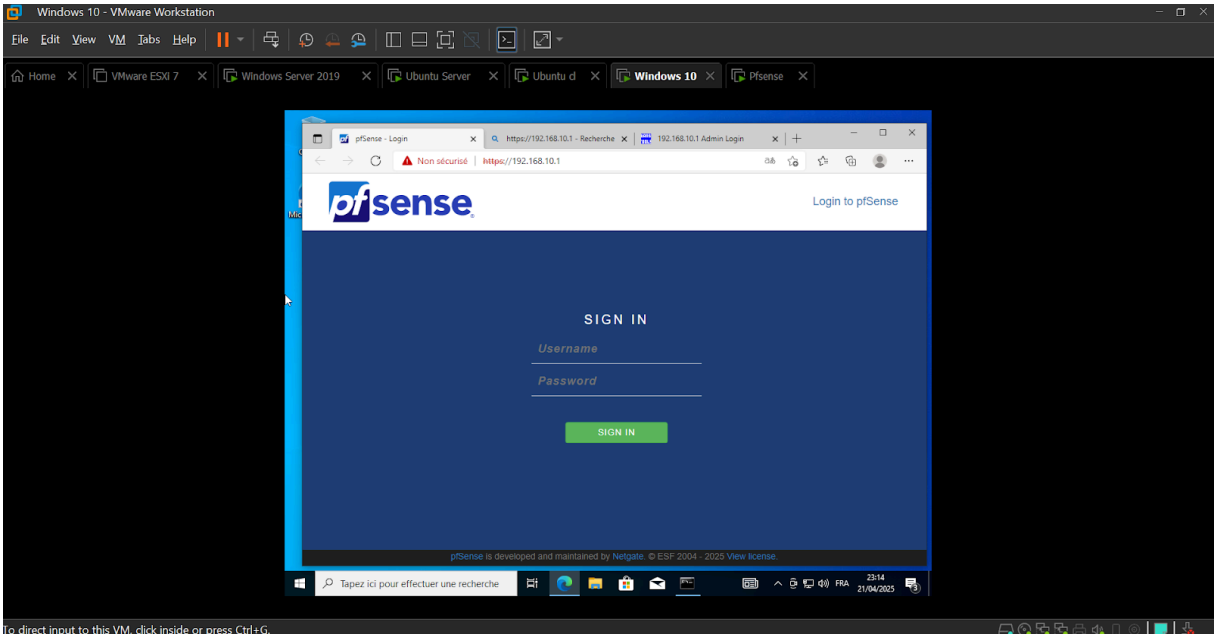

Figure 6 : Interface Web de pfsense

5. Configuration via l'assistant Web

L'assistant WebConfigurator a permis de configurer les éléments suivants :

- Nom d'hôte : firewall.cyberlimited.local
- Domaine local : cyberlimited.local
- Serveurs DNS : par défaut
- Mot de passe admin : modifié pour plus de sécurité
- Paramètres WAN : laissés en DHCP
- Paramètres LAN : confirmés (192.168.10.1/24)
- Mise à jour automatique : désactivée (par souci de stabilité dans l'environnement test)

À la fin de l'assistant, pfSense était prêt à être utilisé comme pare-feu opérationnel.

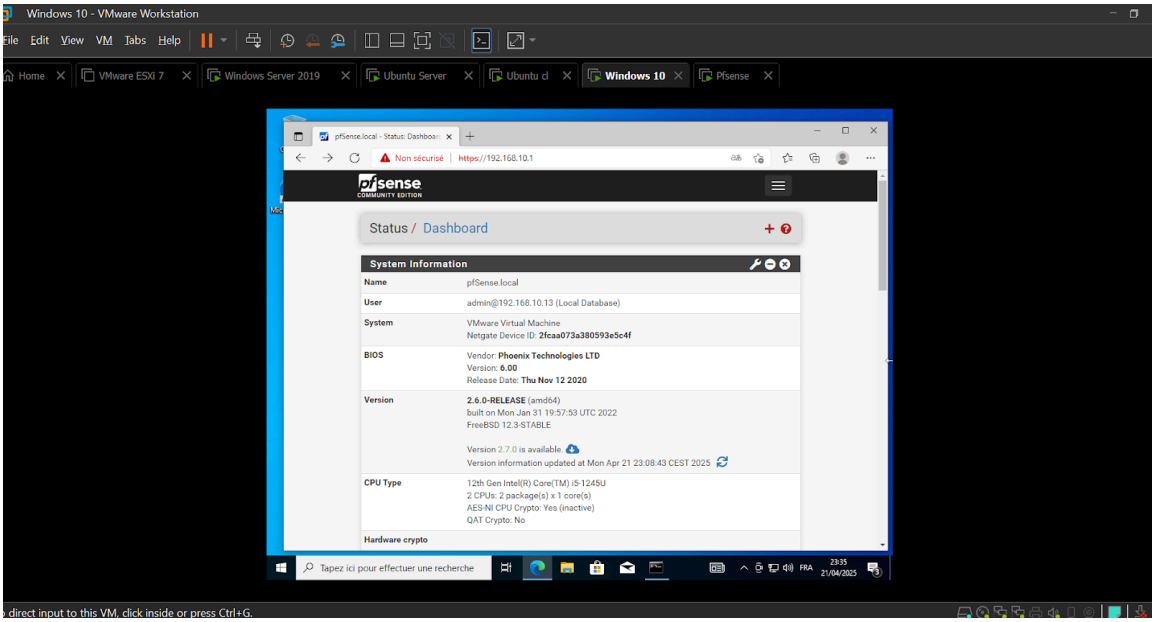

Figure 7 : Tableau de bord pfsense

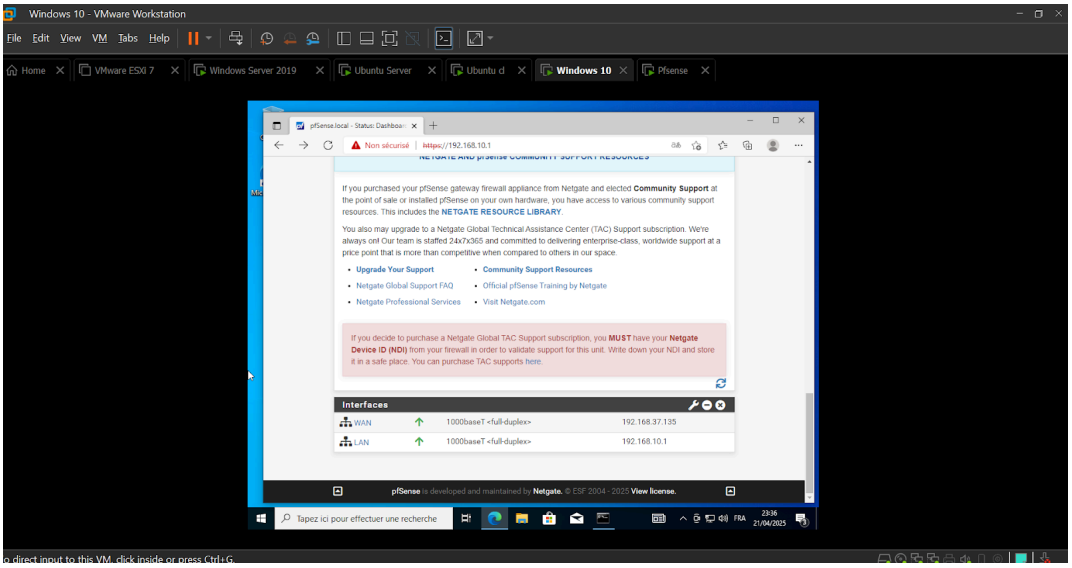

Figure 8 : Résumé des interfaces LAN et WAN

6. Vérification et configuration des règles de pare-feu

Une fois connecté à l'interface, un passage par le menu **Firewall > Rules** a permis d'observer les règles de base créées automatiquement :

- Le **LAN** permet toutes les connexions sortantes vers Internet et les autres réseaux
- Le **WAN** bloque toutes les connexions entrantes (ce qui est correct pour la sécurité)

Des tests simples ont confirmé :

- Que le trafic sortant depuis les clients était autorisé
- Que le trafic entrant depuis l'extérieur était bloqué
- Que le ping entre les machines passait correctement (ICMP autorisé en LAN)

À ce stade, les règles personnalisées n'étaient pas encore nécessaires, mais il était possible d'en ajouter pour filtrer certains ports ou IP spécifiques.

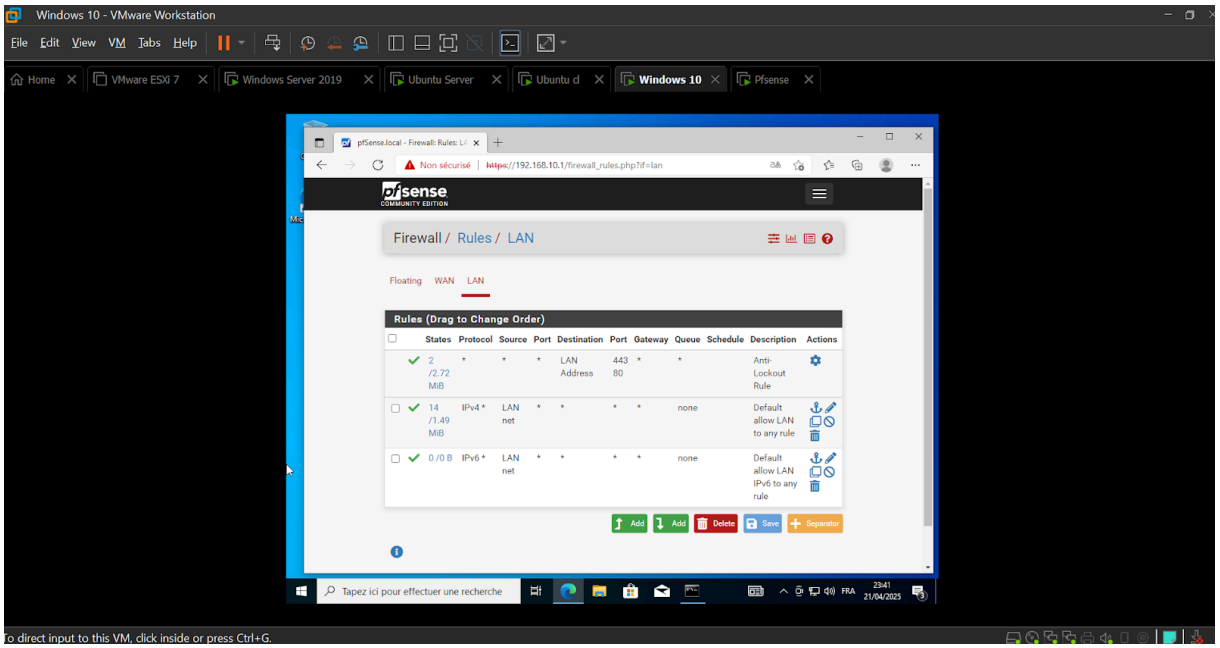

Figure 9 : Règles du pare-feu

7. Limitations et points à surveiller

Même si pfSense est un outil très puissant, plusieurs limites ont été relevées :

- Son interface peut être complexe pour un débutant
- Les erreurs de configuration réseau peuvent bloquer l'accès au système
- L'ajout de services comme Snort peut poser des conflits (cf. Partie 4)

Malgré cela, pfSense reste un outil parfaitement adapté pour l'apprentissage de la sécurité réseau et la gestion centralisée des flux dans un lab.

III. MISE EN PLACE D'UN IDS AVEC SNORT

1. Objectif de l'IDS

Dans un environnement informatique sécurisé, un pare-feu est essentiel mais insuffisant pour détecter certaines attaques internes ou furtives. C'est pourquoi il est nécessaire de déployer un système de détection d'intrusion (**IDS - Intrusion Détection System**) capable d'analyser en profondeur les paquets transitant sur le réseau.
Dans le cadre de ce projet, l'outil **Snort** a été choisi pour remplir ce rôle. Snort est un IDS open source puissant, capable de détecter en temps réel des attaques telles que des scans de ports, des pings excessifs, des anomalies réseau, et plus encore. Il fonctionne en analysant le trafic réseau et en comparant celui-ci à une base de règles de détection.

2. Installation de Snort sur Ubuntu Server

Snort a été installé sur la machine **Ubuntu Server (192.168.10.11)** à l'aide de la commande :

bash
sudo apt install snort -y

Lors de l'installation, deux informations essentielles ont été renseignées :

- L'**interface réseau à surveiller** (ens33)
- Le **réseau interne à surveiller** : 192.168.10.0/24

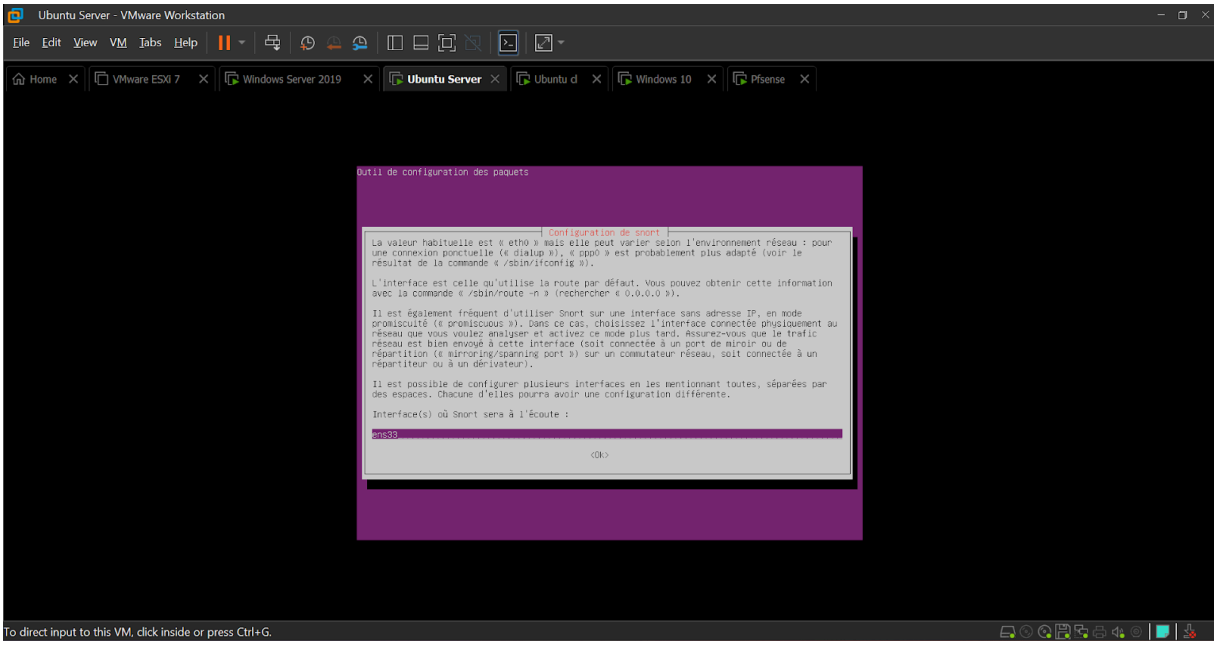

Figure 10 : Installation de snort dans le terminal

3. Configuration manuelle du fichier snort.conf

Suite à un dysfonctionnement initial (fichier de configuration absent), le fichier **/etc/snort/snort.conf** a été recréé manuellement avec les directives de base nécessaires au bon fonctionnement de Snort.

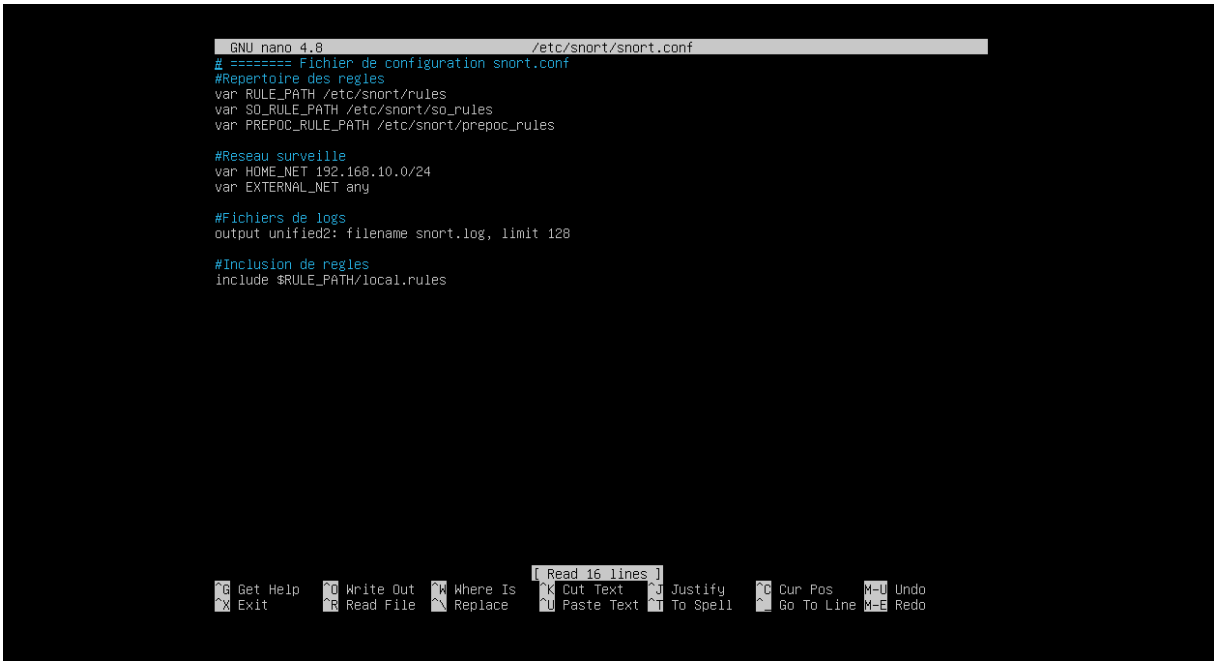

Figure 11 : Fichier de configuration de snort.conf

4. Création d'une règle personnalisée

Un fichier de règles a été ajouté dans /etc/snort/rules/local.rules avec une règle simple permettant de détecter des paquets ICMP (ping) :

alert icmp any -> any (msg:"ICMP detected"; sid:1000001; rev:1;)

Cette règle déclenche une alerte chaque fois qu'un paquet ICMP est détecté sur le réseau.

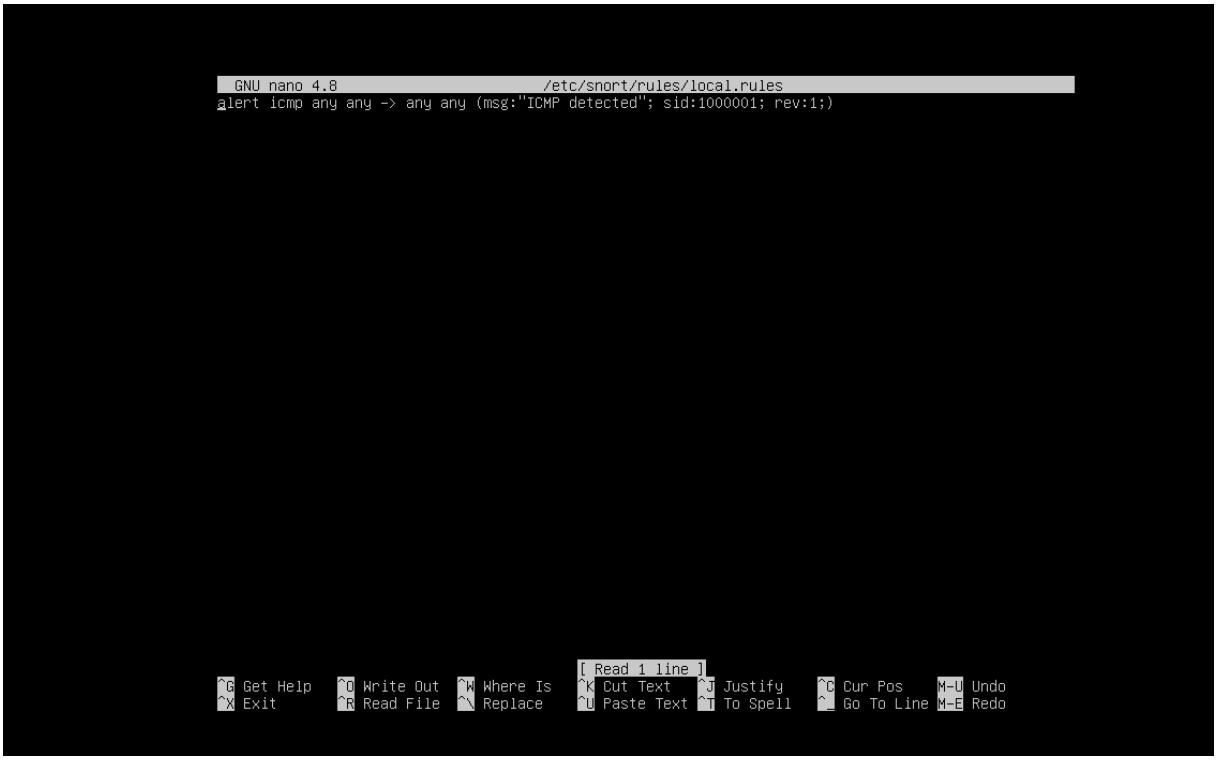

Figure 12 : fichier de configuration des règles personnalisées

5. Vérification de la configuration de Snort

Une commande de test a été utilisée pour valider le fichier de configuration :

sudo snort -T -c /etc/snort/snort.conf

Le message **"Snort successfully validated the configuration!"** a confirmé que Snort était prêt à fonctionner.

```
| none
--------------------------------------------------------------------------------

+-----------------------[rate-filter-config]----------------------------------
| memory-cap : 1048576 bytes
+-----------------------[rate-filter-rules]-----------------------------------
| none
--------------------------------------------------------------------------------

+-----------------------[event-filter-config]---------------------------------
| memory-cap : 1048576 bytes
+-----------------------[event-filter-global]---------------------------------
+-----------------------[event-filter-local]----------------------------------
| none
+-----------------------[suppression]----------------------------------------
| none
--------------------------------------------------------------------------------
Rule application order: activation->dynamic->pass->drop->sdrop->reject->alert->log
Verifying Preprocessor Configurations!

[ Port Based Pattern Matching Memory ]

        --== Initialization Complete ==--

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.7.0 GRE (Build 149)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using libpcap version 1.9.1 (with TPACKET_V3)
           Using PCRE version: 8.39 2016-06-14
           Using ZLIB version: 1.2.11


Snort successfully validated the configuration!
Snort exiting
babou@jo:/etc/snort$
```

Figure 13 : Snort prêt à fonctionner

6. Exécution de Snort et détection d'une attaque simulée

Snort a été lancé en mode console avec la commande :

sudo snort -i ens33 -c /etc/snort/snort.conf -A console

Pendant son exécution, une attaque simulée a été déclenchée depuis la machine **Ubuntu Client (192.168.10.10)** via un simple ping vers le serveur :

ping 192.168.10.11

Résultat : une alerte a été affichée dans la console Snort :

[**] [1:1000001:1] ICMP detected [**]

Cela prouve que l'IDS fonctionne et détecte le trafic conforme à la règle définie.

```
Activities   Terminal   Tue 21:55
babou@ubuntu: ~
File  Edit  View  Search  Terminal  Help
untu5 [5,174 kB]
Fetched 5,353 kB in 8s (677 kB/s)
Selecting previously unselected package libblas3:amd64.
(Reading database ... 112885 files and directories currently installed.)
Preparing to unpack .../libblas3_3.7.1-4ubuntu1_amd64.deb ...
Unpacking libblas3:amd64 (3.7.1-4ubuntu1) ...
Selecting previously unselected package liblinear3:amd64.
Preparing to unpack .../liblinear3_2.1.0+dfsg-2_amd64.deb ...
Unpacking liblinear3:amd64 (2.1.0+dfsg-2) ...
Selecting previously unselected package nmap.
Preparing to unpack .../nmap_7.60-1ubuntu5_amd64.deb ...
Unpacking nmap (7.60-1ubuntu5) ...
Setting up libblas3:amd64 (3.7.1-4ubuntu1) ...
update-alternatives: using /usr/lib/x86_64-linux-gnu/blas/libblas.so.3 to provi
de /usr/lib/x86_64-linux-gnu/libblas.so.3 (libblas.so.3-x86_64-linux-gnu) in au
to mode
Processing triggers for libc-bin (2.27-3ubuntu1) ...
Processing triggers for man-db (2.8.3-2) ...
Setting up liblinear3:amd64 (2.1.0+dfsg-2) ...
Setting up nmap (7.60-1ubuntu5) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
babou@ubuntu:~$ nmap 192.168.10.11

Starting Nmap 7.60 ( https://nmap.org ) at 2025-04-22 21:51 PDT
Note: Host seems down. If it is really up, but blocking our ping probes, try -P
n
Nmap done: 1 IP address (0 hosts up) scanned in 3.10 seconds
babou@ubuntu:~$
```

Figure 14 : Terminal Ubuntu affichant l'attaque simulée avec nmap

```
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
04/23-05:02:16.535692  [**] [1:1000001:1] ICMP detected [**] [Priority: 0] {ICMP} 192.168.10.13 -> 1
92.168.10.11
WARNING: No preprocessors configured for policy 0.
04/23-05:02:16.535780  [**] [1:1000001:1] ICMP detected [**] [Priority: 0] {ICMP} 192.168.10.11 -> 1
92.168.10.13
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
```

Figure 15 : Terminal Snort avec l'alerte ICMP visible

7. Résultat et interprétation

L'installation et la configuration de Snort ont permis de :

- Créer une règle de détection personnalisée
- Tester efficacement la surveillance réseau
- Déclencher et observer une alerte en condition réelle

Le fonctionnement correct de Snort prouve la **capacité de l'environnement à détecter en temps réel des comportements suspects**, comme un excès de trafic ICMP ou des scans.

Malgré un démarrage difficile lié à une configuration incomplète, Snort a pu être remis en service manuellement. L'IDS est désormais opérationnel et permet de renforcer significativement la visibilité sur le réseau local. Cette étape est une illustration concrète de l'intégration d'un système de surveillance dans un projet de cybersécurité.

IV. DURCISSEMENT DES SYSTEMES D'EXPLOITATION

1. Objectif du durcissement

Le durcissement des systèmes consiste à appliquer des **mesures de sécurité proactives** pour réduire la surface d'attaque des machines. Il s'agit notamment de **désactiver les services inutiles**, **limiter les ports ouverts**, **appliquer des règles de pare-feu**, et **renforcer les politiques de mot de passe et d'accès**.
Dans ce projet, chaque machine virtuelle a été analysée et sécurisée individuellement selon son rôle : serveur, client Linux ou Windows.

2. Ubuntu Server (192.168.10.11)

Ubuntu Server est le cœur du système Linux du projet. Il a accueilli l'installation de Snort et a été durci pour prévenir tout accès ou service non autorisé.

➢ Mesures appliquées :

- Mise à jour complète du système :

```
sudo apt update && sudo apt upgrade -y
```

- Activation du pare-feu UFW :

```
sudo ufw enable
sudo ufw default deny incoming
sudo ufw allow ssh
```

- Vérification des ports ouverts :

```
sudo ss -tunlp
```

- Installation et configuration de **Fail2ban** pour bloquer les tentatives SSH :

```
sudo apt install fail2ban -y
```

- Suppression des services inutiles (ex. avahi-daemon, cups)
- Surveillance des logs système

```
Tous les paquets sont à jour.
babou@jo:~$ sudo ufw enable
Firewall is active and enabled on system startup
babou@jo:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
babou@jo:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
babou@jo:~$ sudo ufw allow ssh
Skipping adding existing rule
Skipping adding existing rule (v6)
babou@jo:~$ _
```

Figure 16 : Activation du pare-feu UFW

```
babou@jo:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)

babou@jo:~$
```

Figure 17 : Pare-feu actif

```
babou@jo:~$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /lib/systemd/systemd-sysv
all.
Executing: /lib/systemd/systemd-sysv-install enable fail2ban
babou@jo:~$ sudo systemctl start fail2ban
babou@jo:~$ _
```

*Figure 18 : Installation et configuration de **Fail2ban** pour bloquer les tentatives SSH*

```
babou@jo:~$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /lib/systemd/systemd-sysv-inst
all.
Executing: /lib/systemd/systemd-sysv-install enable fail2ban
babou@jo:~$ sudo systemctl start fail2ban
babou@jo:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2025-04-22 12:27:04 UTC; 10min ago
       Docs: man:fail2ban(1)
   Main PID: 92386 (f2b/server)
      Tasks: 5 (limit: 6567)
     Memory: 12.0M
     CGroup: /system.slice/fail2ban.service
             └─92386 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

avril 22 12:27:04 jo systemd[1]: Starting Fail2Ban Service...
avril 22 12:27:04 jo systemd[1]: Started Fail2Ban Service.
avril 22 12:27:04 jo fail2ban-server[92386]: Server ready
babou@jo:~$
```

*Figure 19 : Installation et configuration réussie de **Fail2ban** pour bloquer les tentatives SSH*

3. Ubuntu Client (192.168.10.10)

Ce poste utilisateur Linux a également été durci afin de simuler un environnement de bureau sécurisé.

➢ Mesures appliquées :

- Mise à jour système
- Activation du pare-feu UFW
- Désactivation des services inutiles (Bluetooth, impression…)
- Nettoyage des utilisateurs inutiles
- Suppression de paquets non nécessaires

4. Windows 10 (192.168.10.13)

Le poste utilisateur Windows a été configuré pour résister aux attaques locales et protéger l'utilisateur.

➢ Mesures appliquées :

- Windows Update appliqué
- Antivirus Windows Defender actif
- Pare-feu Windows activé
- Compte administrateur sécurisé
- Verrouillage automatique de session

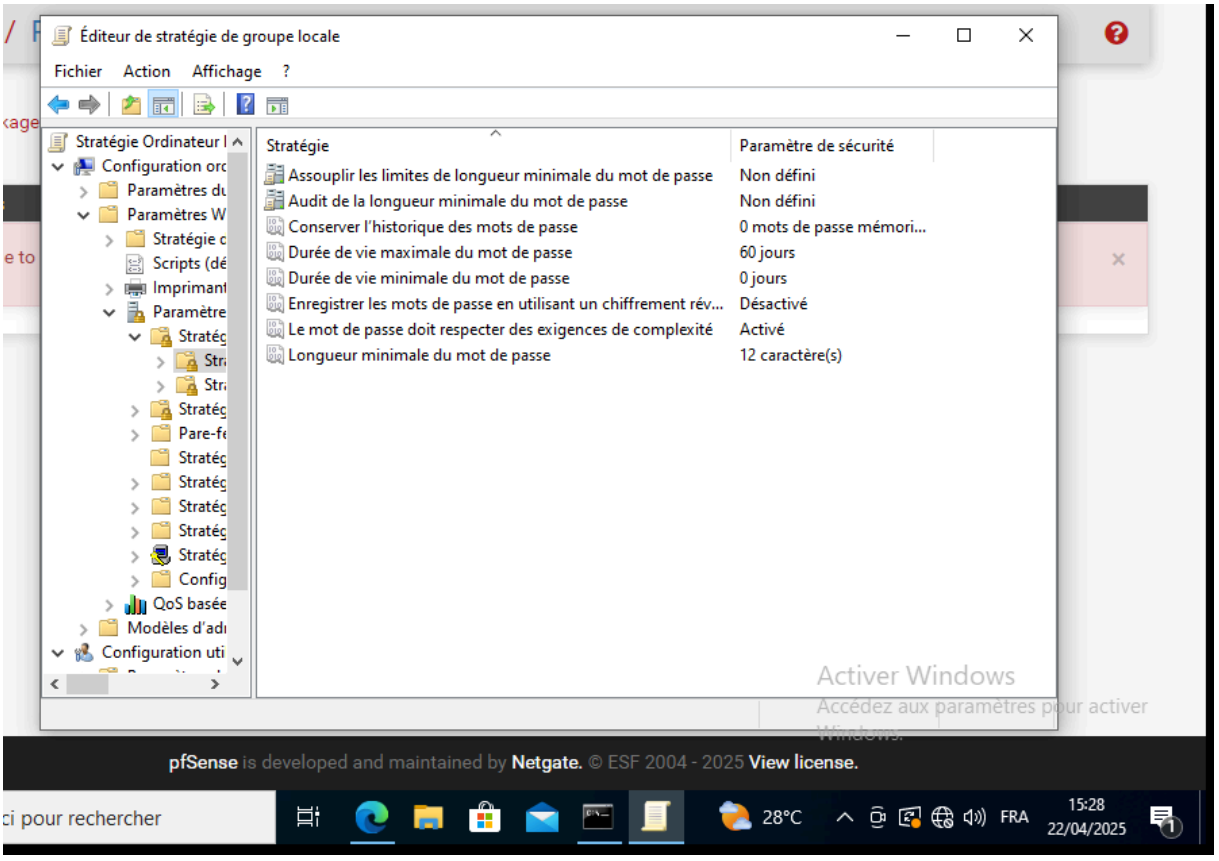

Figure 20 : Paramètre de sécurisation des comptes utilisateur

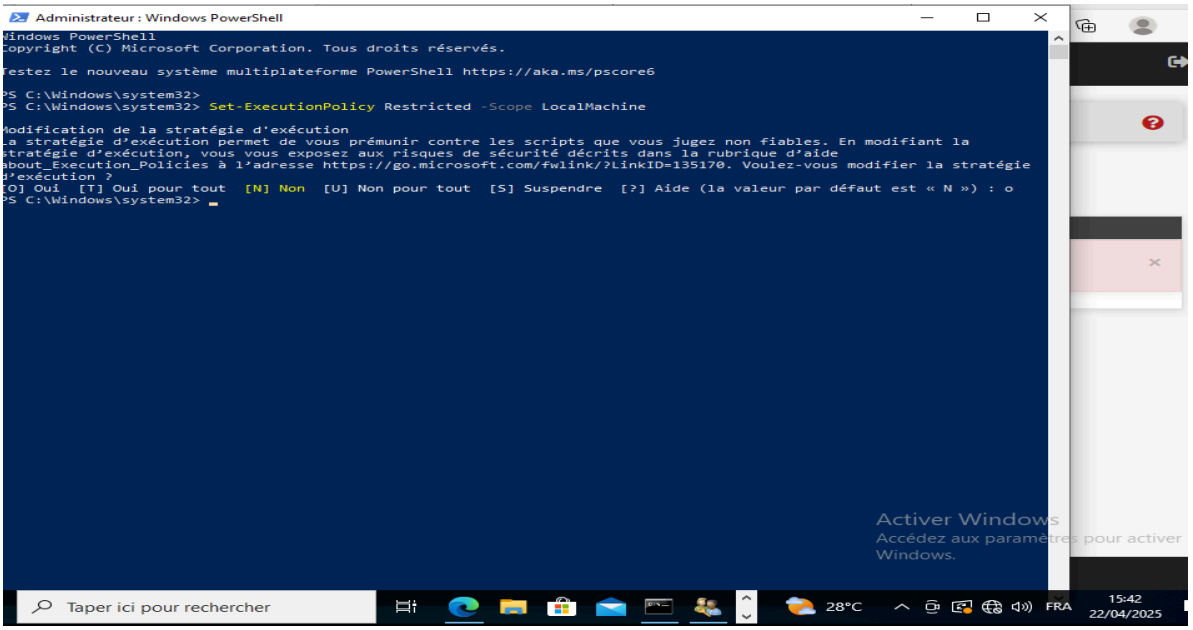

Figure 21 : Technique de protection pour empêcher l'exécution de scripts non signés

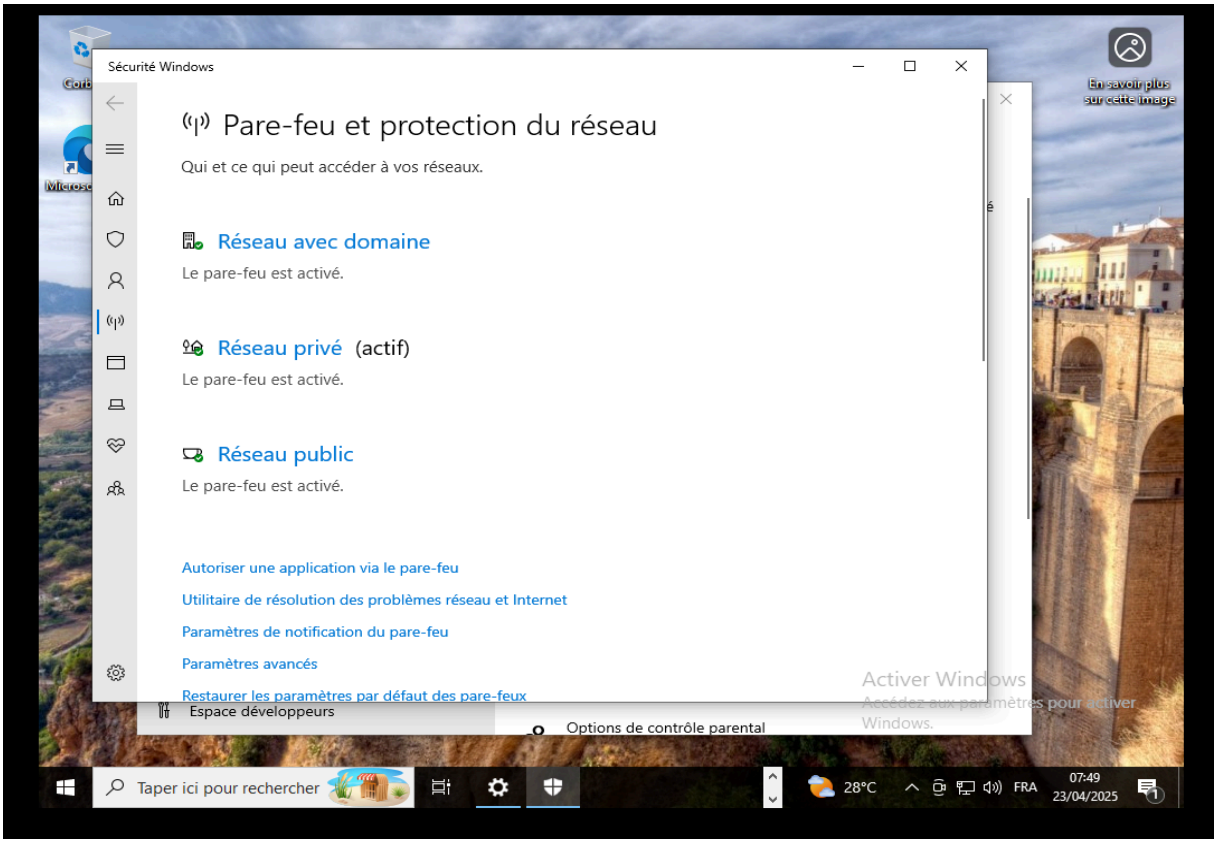

Figure 22 : Antivirus Windows Defender actif et Pare-feu Windows activé

5. Windows Server 2019 (192.168.10.12)

La machine Windows Server a été traitée comme un serveur d'entreprise standard, avec des politiques renforcées.

➢ Mesures appliquées :

- Mise à jour système (Windows Update)
- Configuration des **stratégies de sécurité locales (GPO)** :
  - Mot de passe complexe et expiration
  - Verrouillage après 3 tentatives
- Activation du pare-feu Windows
- Désactivation des services inutiles dans services.msc
- Blocage des ports inutiles (RDP, SMB, etc.)

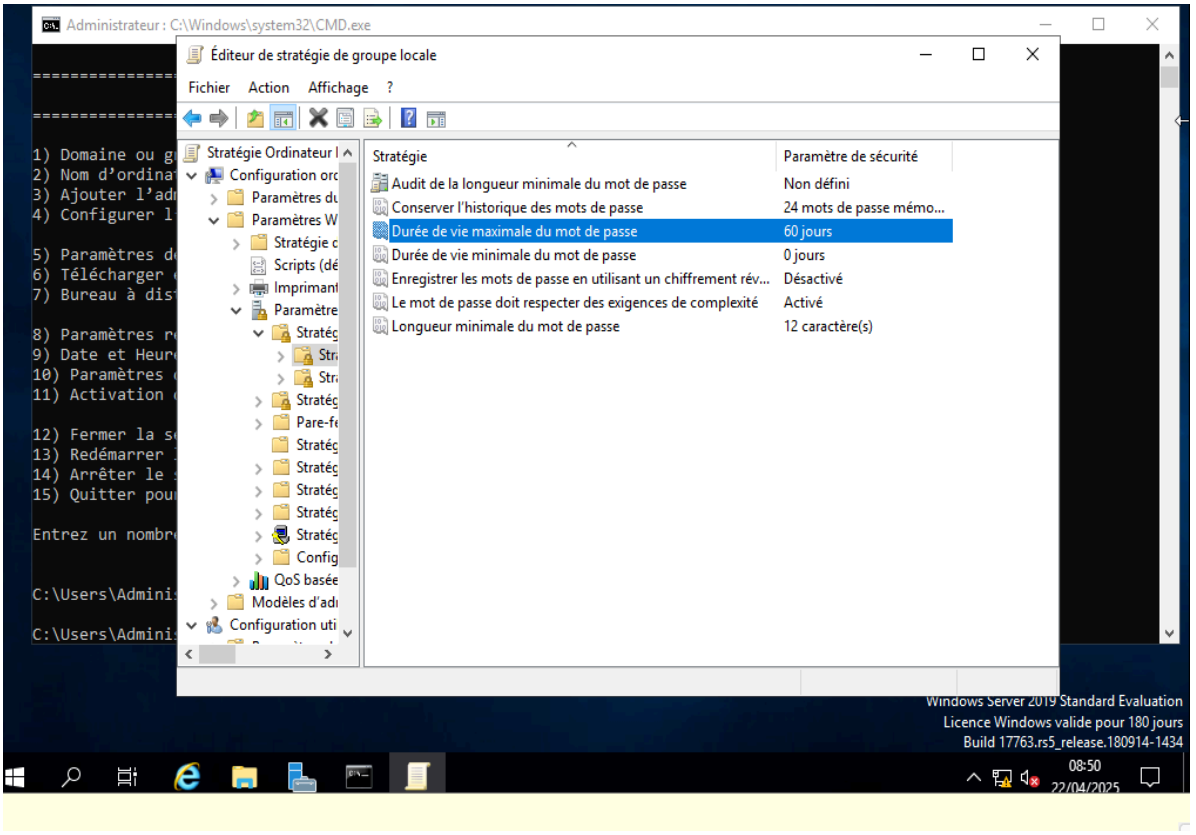

Figure 23 : Paramètre de sécurisation des comptes utilisateur sur Windows server

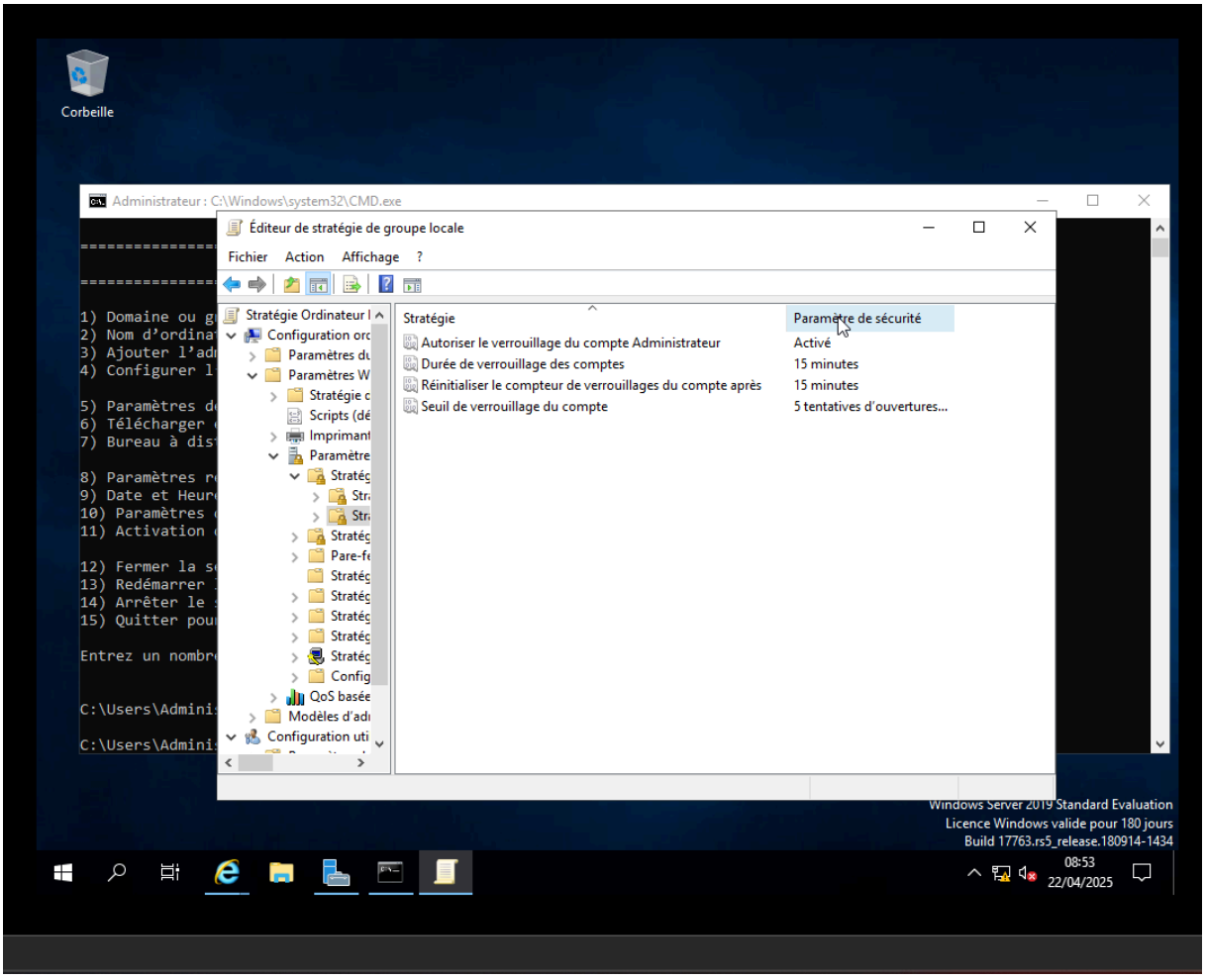

Figure 24 : Paramètre de sécurité

V. ANALYSE ET RETOUR D'EXPERIENCE

1. Ce qui a fonctionné

Le projet a permis de mettre en place avec succès un environnement réseau complet, sécurisé et réaliste. Plusieurs éléments ont bien fonctionné dès les premières étapes :

- La **création et configuration des machines virtuelles** sous VMware s'est déroulée sans incident.
- Le **pare-feu pfSense** a été installé, configuré et utilisé efficacement pour filtrer le trafic réseau.

- Le **durcissement des systèmes Linux et Windows** s'est déroulé conformément aux bonnes pratiques : activation des pare-feu, suppression de services inutiles, mise à jour système, création de politiques de sécurité.
- Les **tests de sécurité** ont permis de valider le bon comportement des protections mises en place (filtrage UFW, blocage fail2ban, détection Snort).

2. Les difficultés rencontrées

Plusieurs obstacles ont été rencontrés au cours du projet, nécessitant des ajustements et des recherches complémentaires :

- L'installation de **Snort sur pfSense** a échoué à cause d'un problème de compatibilité avec la version FreeBSD utilisée.
- L'installation de Snort sur **Ubuntu Server** a d'abord échoué à cause d'une **configuration incomplète** (fichier snort.conf absent, règles manquantes).
- Des **problèmes de réseau** au début ont empêché l'accès à l'interface Web de pfSense jusqu'à ce que les IP et interfaces soient correctement définies.
- Certains services Windows ont été difficiles à identifier pour désactivation sans impacter le système.

3. Solutions et ajustements apportés

Face aux difficultés, plusieurs solutions ont été mises en œuvre avec succès :

- **Création manuelle du fichier snort.conf** à partir des bonnes pratiques de Snort 2.9, et ajout d'une règle personnalisée.
- Test de Snort en mode console pour simplifier le processus de détection.
- Réorganisation de la configuration réseau dans VMware pour garantir une connectivité LAN stable.
- Recherche et documentation des services essentiels sous Windows pour ne pas désactiver de composants critiques.

Ces correctifs ont permis d'**avancer dans le projet malgré les blocages**, et ont favorisé un apprentissage pratique solide.

4. Limites du POC

Ce POC (Proof of Concept) reste une démonstration à échelle réduite. Parmi les limites :

- Aucune centralisation des logs n'a été mise en place (ex. SIEM).
- Les attaques simulées restent simples (ICMP, scan de ports).
- Snort a été utilisé de façon basique, sans mise à jour de base de règles communautaires.
- Les rôles Windows (ex : Active Directory) n'ont pas été explorés.

Cependant, l'objectif principal qui était de **concevoir un environnement sécurisé avec supervision réseau et test de défenses** a été atteint.

CONCLUSION

Ce projet a permis de concevoir un environnement réseau sécurisé complet, simulant l'infrastructure d'une entreprise à travers la mise en place de machines virtuelles connectées via un pare-feu pfSense, le durcissement de systèmes Linux et Windows, l'intégration d'un IDS (Snort) et la réalisation de tests de sécurité réels. Malgré certaines difficultés techniques, notamment lors de l'installation de Snort, des solutions efficaces ont été mises en œuvre pour parvenir à un système fonctionnel et sécurisé. Ce travail a permis de mettre en pratique des compétences essentielles en cybersécurité, comme la gestion des flux réseau, la détection d'intrusions, le contrôle d'accès et la configuration de défenses actives. Il constitue une excellente base de préparation au monde professionnel et pourra être enrichi à l'avenir par l'ajout d'une supervision centralisée, de scénarios d'attaque plus poussés, ou de services d'annuaire. Ce projet représente une étape concrète vers une meilleure compréhension de la cybersécurité d'entreprise.